

Sustainable by Construction: Embedding Sustainability Constraints into Specification-Driven Software Development

Ravi Kumar Kappagantu¹

Abstract: Green software engineering has traditionally emphasised post-development optimisation techniques such as infrastructure tuning, runtime efficiency improvements, and cloud resource management. The rapid acceleration of AI-assisted development introduces a new challenge: computational inefficiencies can now propagate at unprecedented scale and velocity. This paper proposes a specification-driven sustainability framework, GreenSpec, that embeds green software constraints directly into the software development lifecycle. Rather than treating sustainability as an afterthought enforced through manual review or operational tuning, GreenSpec integrates machine-interpretable sustainability constraints into executable development specifications, enabling sustainable software to emerge by construction. A Python-specific sustainability profile, GreenSpec-Py, is presented as a practical implementation exhibit. The framework aligns with UN Sustainable Development Goals 9, 12, and 13.

Keywords: Green Software Engineering, Specification-Driven Development, AI-Assisted Development, Software Governance, Policy-as-Code, Green Coding

1 Introduction

Software systems contribute increasingly to global energy consumption through expanding computational workloads, inefficient data processing, excessive infrastructure utilisation, and large-scale AI model execution. While significant research has addressed infrastructure efficiency, cloud resource allocation, and runtime energy management [VSC23, Ra23], comparatively little attention has been directed toward preventing inefficiencies during software creation itself.

The emergence of AI-assisted software development amplifies this concern. Generative AI systems can produce large volumes of code rapidly, yet they may also replicate inefficient patterns, introduce unnecessary abstractions, embed redundant data movement, and encourage resource-intensive architectures. In enterprise contexts, such inefficiencies scale across thousands of services and millions of transactions, producing cumulative environmental impact [Sc15].

Existing green coding approaches rely primarily on developer education, coding guidelines, and manual review processes. These remain reactive rather than preventative. This paper proposes a fundamentally different perspective: sustainability constraints should be embedded directly into specification-driven development processes such that software systems become sustainable by construction.

The proposed framework, GreenSpec, introduces machine-interpretable sustainability specifications integrated into software design, code generation, validation, and governance pipelines. A Python-specific sustainability profile, GreenSpec-Py, is introduced as a practical exhibit; its full constraint inventory is presented in the Appendix.

2 Background and Related Work

2.1 Green Software Engineering

Green software engineering research has concentrated on energy-efficient infrastructure, cloud optimisation, hardware-aware scheduling, carbon-aware workload placement, and runtime telemetry analysis [VSC23, Ra23, GS24]. These approaches improve operational efficiency but address inefficiencies after software has been implemented. Common sources of waste, including excessive memory allocation, redundant database queries, unnecessary API calls, inefficient serialisation, and over-logging, frequently originate during design and implementation phases [Na11].

2.2 AI-Assisted Development and Sustainability Risk

AI-assisted development introduces additional sustainability risks. Code generation systems optimise for functional correctness but may propagate duplicated abstractions, over-engineered service decomposition, excessive dependency inclusion, and computationally expensive logic patterns. Without governance controls, AI-generated software may inadvertently negate efficiency gains achieved at the infrastructure level [Sc15]. Embedding computation for large language models can introduce significant GPU energy consumption at scale [LVL23], making AI workload governance an integral dimension of green software practice.

2.3 Specification-Driven Development

Specification-driven development shifts software engineering from imperative implementation toward declarative system definition. Specifications increasingly drive API generation, infrastructure provisioning, test synthesis, validation pipelines, and AI-assisted code generation. This shift presents an opportunity to embed sustainability policies directly into executable specifications, applying governance constraints upstream rather than downstream.

3 Problem Statement and Research Questions

Current software sustainability approaches lack: (1) machine-enforceable sustainability governance; (2) integration with AI-assisted development workflows; (3) specification-

level sustainability controls; (4) language-specific sustainability profiles; and (5) automated sustainability validation pipelines.

This paper investigates three research questions.

RQ1: Can sustainability constraints be embedded into specification-driven development workflows in a machine-enforceable way?

RQ2: Can language-specific sustainability profiles improve software efficiency through automated enforcement?

RQ3: Can AI-assisted development pipelines enforce sustainability policies during code generation?

4 The GreenSpec Framework

4.1 Architectural Overview

GreenSpec introduces a sustainability governance layer integrated into specification-driven software engineering pipelines. The framework comprises five components: (1) a Sustainability Constraint Layer; (2) Language-Specific Profiles; (3) AI-Assisted Development Governance; (4) Static Enforcement Pipelines; and (5) a Runtime Telemetry Feedback loop.

4.2 Sustainability Constraint Layer

The Sustainability Constraint Layer defines machine-readable policies governing computational efficiency, memory utilisation, network behaviour, data movement, storage consumption, logging behaviour, and AI workload optimisation. These constraints function analogously to policy-as-code frameworks used in security and compliance governance [GS24]. A representative constraint is expressed as:

```
constraint:
  id: GS-IO-001
  category: NetworkEfficiency
  rule: Avoid synchronous chatty service calls
  enforcement: static-analysis
  severity: high
```

Each constraint carries an identifier, a category, a plain-language rule, an enforcement mechanism, and a severity classification. Constraints are consumed by static analysis tools, CI/CD pipeline gates, and AI code generation contexts.

4.3 Language-Specific Profiles and AI Governance

GreenSpec supports extensible language-specific sustainability profiles. The initial profile, GreenSpec-Py, covers Python and is fully specified in the Appendix. AI-assisted code generation systems can consume sustainability-aware specifications during prompt construction, retrieval augmentation, and validation stages, enabling sustainability constraints to govern generated code automatically. A representative prompt augmentation is: Generate Python code optimised for minimal memory allocation, async I/O patterns, batched database access, generator-based iteration, and reduced serialisation overhead.

4.4 Runtime Telemetry Feedback

Runtime telemetry enables continuous refinement of sustainability policies through CPU profiling, memory telemetry, energy estimation, API utilisation metrics, and database access analytics. This creates a closed-loop governance model: empirical runtime evidence informs constraint updates, which are propagated back into specifications, enforcement pipelines, and AI generation contexts.

5 Green Spec-Py: Python Sustainability Profile

Python presents unique sustainability challenges due to dynamic typing overhead, interpreter-level inefficiencies, high memory allocation rates, and frequent misuse of data structures. GreenSpec-Py formalises sustainability constraints across four primary dimensions; the full specification including code examples is provided in the Appendix.

Memory Efficiency (GS-PY-MEM-001): Generator expressions must be preferred over list comprehensions for large dataset iteration. The expression `sum(x for x in dataset)` avoids materialising an intermediate list, reducing peak memory allocation. Static analysis rules detect list comprehensions passed directly to aggregation functions and flag them for replacement.

I/O Efficiency (GS-PY-IO-001): Asynchronous I/O patterns must be applied to all network-bound operations. Synchronous calls block threads unnecessarily, increasing idle resource consumption. The constraint mandates `async/await` patterns for HTTP, database, and file I/O operations in service contexts.

Database Efficiency (GS-PY-DB-001): $N+1$ query patterns, in which a query is issued inside a loop iterating over a prior result set, represent a significant source of database infrastructure overhead. GreenSpec-Py requires batched retrieval, eager loading, and query aggregation strategies.

AI Workload Efficiency (GS-PY-AI-001): Redundant embedding generation is prohibited. Embedding computation can introduce significant GPU energy consumption at scale [LVL23]; embedding cache layers, vector deduplication, and semantic reuse strategies must be applied wherever embeddings are generated from recurring or similar input content.

6 Enforcement Architecture

GreenSpec constraints are enforced through three complementary mechanisms. Static analysis integrates with AST analysis, linting frameworks, and policy engines. It is important to distinguish GreenSpec from traditional linting: tools such as `pylint`, `Ruff`, and `Bandit` primarily target correctness, style consistency, or security concerns. GreenSpec extends this paradigm by introducing sustainability-aware architectural constraints that govern computational resource behaviour across development, deployment, and AI-assisted code generation workflows, addressing a dimension that established static analysis tooling does not cover.

Specification validation embeds sustainability constraints into OpenAPI specifications, AsyncAPI definitions, GraphQL schemas, Infrastructure-as-Code templates, and AI orchestration workflow definitions. AI development integration provides sustainability prompts, policy retrieval context, architecture constraints, and validation feedback to AI coding assistants, ensuring that code synthesis operates under the same constraint regime as human-authored contributions.

6.1 Prototype Enforcement Example

To demonstrate operational feasibility, a prototype AST rule was implemented for constraint GS-PY-MEM-001. The rule traverses the abstract syntax tree of a Python source module and identifies patterns in which a list comprehension is passed directly as the sole argument to an aggregation call such as `sum()`, `min()`, or `max()`. Preliminary testing across a representative set of twelve service-layer utility modules drawn from an open-source Python microservices reference implementation demonstrated that the rule could automatically identify memory-inefficient iteration patterns and surface them as structured enforcement findings consumable by CI/CD pipeline gates. No automatic code modifications were applied. This conceptual proof-of-feasibility confirms that GreenSpec constraints can be translated into executable, context-sensitive static analysis rules integrated into standard development workflows.

7 Comparative Assessment

Tab. 1 summarises the governance capabilities of GreenSpec relative to traditional green coding approaches across six dimensions.

Capability	Traditional	GreenSpec
Manual Governance Burden	High	Low
AI Integration	Limited	Native
Automated Enforcement	Partial	Extensive
Specification-Level Control	Minimal	Strong

Capability	Traditional	GreenSpec
Runtime Feedback	Moderate	Integrated
Language Extensibility	Weak	Strong

Tab. 1: GreenSpec compared to traditional green coding approaches

The framework demonstrates structural advantages in automated enforcement and AI integration; two dimensions largely absent from traditional green coding practice. Manual governance overhead is substantially reduced by embedding sustainability constraints into specification and pipeline artefacts rather than relying on developer awareness and peer review.

8 Discussion

The proposed approach reframes sustainable software engineering as an architectural governance problem rather than merely a runtime optimisation exercise. This shift is of increasing importance as AI-assisted development accelerates software production velocity, expands code volume, and enables inefficient patterns to propagate at industrial scale.

Embedding sustainability constraints into executable specifications provides several strategic advantages: earlier prevention of inefficiencies, automated governance without manual reviewer burden, reduced operational overhead, and scalable enforcement that applies equally to human-authored and AI-generated code. The framework aligns with broader enterprise governance trends including policy-as-code, platform engineering, and DevSecOps, positioning sustainability enforcement within the same architectural governance layer as security and compliance controls.

The proposed framework aligns with UN Sustainable Development Goal 13 (Climate Action) by addressing energy consumption of software systems at the point of creation, with SDG 9 (Industry, Innovation and Infrastructure) by embedding sustainable innovation practices into software engineering workflows, and with SDG 12 (Responsible Consumption and Production) by governing the computational resources consumed during both software development and operation.

9 Limitations and Future Work

GreenSpec is presented as a conceptual contribution supported by a profile exhibit and prototype rule. Quantitative empirical evaluation of energy savings, developer adoption overhead, and false-positive rates for static analysis rules represents the primary direction for future work. Additional research directions include formal sustainability ontologies, carbon-aware AI code generation, runtime energy telemetry integration, sustainability scoring models, and multi-language policy frameworks. Further work may explore integration with large language model orchestration systems, agentic software engineering platforms, and autonomous development pipelines.

10 Conclusion

Green software engineering must evolve beyond post-development optimisation into specification-level architectural governance. The rise of AI-assisted development increases both the scale and speed at which inefficient software patterns can proliferate. The GreenSpec framework demonstrates how sustainability constraints can be embedded into specification-driven development pipelines, enabling software systems to become sustainable by construction. The GreenSpec-Py profile illustrates how language-level constraints can be formalised for practical enforcement. The proposed approach establishes a foundation for sustainability-aware software engineering ecosystems in which environmental efficiency is an intrinsic property of software architecture rather than a retrospective optimisation concern.

References

- [VSC23] Verdecchia, Roberto; Sallou, June; Cruz, Luís: A Systematic Review of Green AI. *WIREs Data Mining and Knowledge Discovery* 13(4), e1507, 2023. DOI: 10.1002/widm.1507
- [Ra23] Radovanovic, Ana et al.: Carbon-Aware Computing for Datacenters. *IEEE Transactions on Power Systems* 38(2), pp. 1270–1280, 2023. DOI: 10.1109/TPWRS.2022.3173250
- [Na11] Naumann, Stefan et al.: The GREENSOFT Model: A Reference Model for Green and Sustainable Software and its Engineering. *Sustainable Computing: Informatics and Systems* 1(4), pp. 294–304, 2011. DOI: 10.1016/j.suscom.2011.06.004
- [Sc15] Sculley, D. et al.: Hidden Technical Debt in Machine Learning Systems. In: *Advances in Neural Information Processing Systems* 28 (NeurIPS), pp. 2503–2511, 2015. <https://proceedings.neurips.cc/paper/2015/hash/86df7dcfd896fcdf2674f757a2463eba-Abstract.html>
- [GS24] Green Software Foundation: Software Carbon Intensity (SCI) Specification, v1.0. <https://sci.greensoftware.foundation>, accessed: May 2025
- [LVL23] Luccioni, Alexandra Sasha; Viguier, Sylvain; Ligozat, Anne-Laure: Estimating the Carbon Footprint of BLOOM, a 176B Parameter Language Model. *Journal of Machine Learning Research* 24(253), pp. 1–15, 2023. <https://jmlr.org/papers/v24/23-0069.html>

¹ Ravi Kumar Kappagantu - contact: Lloyds Technology Centre, ravi-kumar.kappagantu@lloydsbanking.com

Appendix: GreenSpec-Py Sustainability Specification

This appendix presents the complete GreenSpec-Py constraint specification for the Python ecosystem. Each constraint is defined with its identifier, category, severity, enforcement mechanism, rationale, and compliant/non-compliant code examples. Tab. 2 provides the master constraint inventory.

ID	Category	Rule Summary	Severity	Enforcement
GS-PY-MEM-001	Memory	Prefer generators over list comprehensions in aggregations	High	AST / Lint
GS-PY-IO-001	I/O	Use async/await for all network-bound operations	High	AST / Lint
GS-PY-DB-001	Database	Prevent N+1 query patterns; enforce batched retrieval	High	AST / ORM
GS-PY-LOG-001	Logging	Restrict debug-level logging in production code paths	Medium	AST / Lint
GS-PY-AI-001	AI / GPU	Cache and reuse embeddings; prohibit redundant generation	High	Code Review

Tab. 2: GreenSpec-Py master constraint inventory

A.1 GS-PY-MEM-001 — Memory Efficiency: Generator Preference

Constraint: List comprehensions passed directly to aggregation functions (sum, min, max, any, all, sorted) must be replaced with semantically equivalent generator expressions. Rationale: List comprehensions fully materialise an intermediate list in heap memory before the aggregation function consumes it. Generator expressions evaluate elements lazily, reducing peak memory allocation by O(n) and decreasing garbage collection pressure in long-running service processes.

Non-compliant:

```
total = sum([x * 2 for x in large_dataset])
result = max([item.score for item in records])
```

Compliant:

```
total = sum(x * 2 for x in large_dataset)
result = max(item.score for item in records)
```

Enforcement: AST rule detects ast.ListComp as the sole positional argument to any of the above aggregation calls. Violation severity: High. CI/CD gate: fail.

A.2 GS-PY-IO-001 — I/O Efficiency: Asynchronous Network Operations

Constraint: All network-bound I/O operations in service-layer modules must use async/await patterns. Synchronous blocking calls are prohibited in request-handling code paths. Rationale: Synchronous I/O blocks the calling thread for the full duration of the

network round trip. In service contexts, thread-blocking operations increase idle CPU time, inflate thread pool size requirements, and elevate infrastructure resource consumption under concurrent load.

Non-compliant:

```
def fetch_data(url):
    response = requests.get(url)
    return response.json()
```

Compliant:

```
async def fetch_data(url):
    async with session.get(url) as response:
        return await response.json()
```

Enforcement: AST rule identifies synchronous HTTP library calls (`requests.get`, `requests.post`, `urllib.request`) in functions not declared `async`. Violation severity: High. CI/CD gate: fail.

A.3 GS-PY-DB-001 — Database Efficiency: N+1 Query Prevention

Constraint: Database queries must not be issued inside iteration loops over prior query result sets. Batched retrieval, eager loading (ORM `select_related` / `prefetch_related`), and query aggregation strategies must be applied. Rationale: The N+1 query pattern issues one query to retrieve N parent records, then N additional queries to retrieve related data for each record. This produces N+1 database round trips where a single batched query would suffice, amplifying database server load linearly with result set size.

Non-compliant:

```
orders = Order.objects.all()
for order in orders:
    print(order.customer.name) # N+1: one query per order
```

Compliant:

```
orders = Order.objects.select_related('customer').all()
for order in orders:
    print(order.customer.name) # single JOIN query
```

Enforcement: AST and ORM analysis rules detect attribute access on related model fields inside for-loops over ORM querysets without `select_related` or `prefetch_related`. Violation severity: High. CI/CD gate: fail.

A.4 GS-PY-LOG-001 — Logging Efficiency: Production Debug Logging

Constraint: `logging.debug()` calls must not appear in production code paths without a guard checking the effective log level. Unconditional debug logging in hot paths is prohibited. Rationale: Debug-level logging in production increases storage consumption, network transmission volume, and observability infrastructure load. In high-throughput services,

unchecked debug logging can produce gigabytes of log data per hour, amplifying infrastructure costs across storage, ingestion, and indexing pipelines.

Non-compliant:

```
def process_request(req):
    logging.debug(f'Processing: {req}')
    return handle(req)
```

Compliant:

```
def process_request(req):
    if logger.isEnabledFor(logging.DEBUG):
        logger.debug(f'Processing: {req}')
    return handle(req)
```

Enforcement: AST rule flags unconditional `logging.debug()` calls in functions that are not themselves test or configuration utilities. Violation severity: Medium. CI/CD gate: warn.

A.5 GS-PY-AI-001 — AI Workload Efficiency: Embedding Cache Reuse

Constraint: Embedding generation calls must be wrapped in a cache layer. Redundant embedding generation for identical or semantically equivalent inputs is prohibited. Rationale: Large language model embedding generation is computationally expensive, typically requiring GPU inference per call. In applications that repeatedly embed similar or identical text inputs, such as document retrieval pipelines and chatbot context builders, uncached embedding generation can consume GPU-hours that a cache hit would reduce to a nanosecond memory lookup.

Non-compliant:

```
def get_embedding(text):
    return model.encode(text)  # always calls model
```

Compliant:

```
_cache = {}
def get_embedding(text):
    if text not in _cache:
        _cache[text] = model.encode(text)
    return _cache[text]
```

Enforcement: Code review gate and AST pattern matching identify `model.encode()` / embedding API calls outside of cache-wrapping functions. Production deployments should use a persistent vector store or dedicated embedding cache service rather than an in-process dictionary for scalability. Violation severity: High. Gate: code review required.